

MobileSec : Un Analyseur de Sécurité Multi-Couches pour l'Authentification et la Gestion de Session dans les Applications Android

Hafssa Chaoulid^a, Iliass Chbani^a, Jihad Dahouas^a, Sayf Eddine Laamri^a, Mohamed Lachgar^a

^a*Université Cadi Ayyad, École Nationale des Sciences Appliquées, Département Informatique, Marrakech, Maroc*

Abstract

Contribution logicielle. MobileSec est un framework open-source d'analyse de sécurité automatisée ciblant les vulnérabilités d'authentification et de gestion de session dans les applications Android. Il combine la décompilation statique via JADX [6], les tests dynamiques d'API, l'inspection des cookies HTTP conforme à l'OWASP [2], l'analyse de session avancée (durée de vie des tokens, rotation, fixation de session au sens du CWE-384 [13]), et une assistance à la remédiation propulsée par l'API Claude [8].

Contribution scientifique. MobileSec comble un manque identifié dans les outils existants [3] en proposant une couverture spécialisée du cycle de vie des tokens JWT [10, 17] et des tests de fixation de session automatisés, deux domaines sous-couverts par les frameworks d'analyse mobile actuels [14, 15].

Protocole d'évaluation. Sur les benchmarks InsecureBankv2 [4] et DVBA [5], MobileSec a détecté 148 findings répartis en six domaines dans un environnement contrôlé (Python 3.13, JADX 1.5.5, Windows 10, 16 Go RAM). L'outil atteint un rappel de 0,92 et une précision de 0,36, cette dernière reflétant des faux positifs provenant des bibliothèques tierces — une limite connue discutée en Section 6. Il génère des rapports JSON structurés et des rapports PDF avec des checklists MASVS [2], incluant des critères d'acceptation au format Gherkin par type d'authentification.

Keywords: Sécurité Android, Analyse statique, Analyse dynamique, Authentification, Gestion de session, MASVS, OWASP, JWT, Session Fixation, Token Lifetime, Détection de vulnérabilités

Metadata

TABLE 1 – Métadonnées du code (obligatoire)

Nr.	Description	Information
C1	Version actuelle du code	v2.0
C2	Lien permanent vers le dépôt GitHub	https://github.com/IliassCyber/mini-projet-android
C3	Licence du code	MIT License
C4	Système de versionnage	Git
C5	Langages, outils et services	Python 3.13, JADX 1.5.5, Flask 3.0, mitmproxy 10.2, API Anthropic Claude (claude-sonnet-20240229), PyJWT 2.8
C6	Prérequis et dépendances	Windows 10 / Ubuntu 22.04 LTS, Python 3.10–3.13, JADX dans le PATH, pip : <code>fpdf2≥2.7</code> , <code>anthropic≥0.21</code> , <code>colorama</code> , <code>requests</code> , <code>python-dotenv</code> , <code>flask</code> , <code>mitmproxy</code> , <code>PyJWT</code>
C7	Lien vers la documentation	https://github.com/IliassCyber/mini-projet-android/blob/main/README.md
C8	Tag de release / commit hash	v2.0 / a3f7c12 (branche <code>main</code> , analysé le 17/05/2026)

1. Motivation et Significance

La sécurité des applications mobiles est devenue une préoccupation critique alors que les smartphones gèrent des opérations de plus en plus sensibles — bancaires, médicales, identitaires. Android, dont la part de marché dépasse 70% des systèmes d’exploitation mobiles [1], constitue une surface d’attaque prioritaire. Les faiblesses d’authentification et la gestion inappropriée des sessions figurent systématiquement parmi les vulnérabilités les plus exploitées dans les applications mobiles [2].

Lacunes des outils existants

MobSF fournit une large couverture de la surface d’attaque Android mais n’implémente pas de tests spécifiques au cycle de vie des tokens JWT [10, 17], ne vérifie pas la fixation de session au sens du CWE-384 [13], et ne propose pas de remédiation guidée par un modèle de langage. Yaazhini se concentre sur l’OWASP Mobile Top 10 mais reste limité à l’analyse statique, sans tests dynamiques de session.

Contribution de MobileSec

MobileSec comble ce manque en proposant un pipeline automatisé et unifié, spécifiquement conçu autour de la sécurité de l’authentification et des sessions, avec : (1) l’intégration dans un seul outil de l’analyse statique (JADX, patterns MASVS), des tests dynamiques d’API, de l’inspection des cookies OWASP, de l’audit du stockage Android et de l’analyse avancée du cycle de vie des tokens ; (2) la couverture explicite des deux paradigmes d’authentification prévalents (JWT pour DVBA, cookies de session pour InsecureBankv2) ; (3) la génération de critères d’acceptation structurés Gherkin et de checklists MASVS assistées par IA.

2. Description du Logiciel

2.1. Architecture du Logiciel

MobileSec est structuré en un pipeline séquentiel de sept étapes, chaque étape étant implémentée comme un module Python indépendant. Cette conception modulaire permet de remplacer ou d’étendre les composants individuels sans affecter le pipeline global.

La Figure 1 présente l’architecture complète du système MobileSec v2.0 avec ses sept modules organisés en trois grandes zones : l’analyse (modules 1–5), le traitement IA (module 6), et la génération de sorties (module 7). Les flux de données inter-modules sont matérialisés par des flèches nommées ; les artefacts de sortie (JSON, PDF) et les dépendances externes (JADX, mitmproxy, API Claude) sont explicitement représentés.

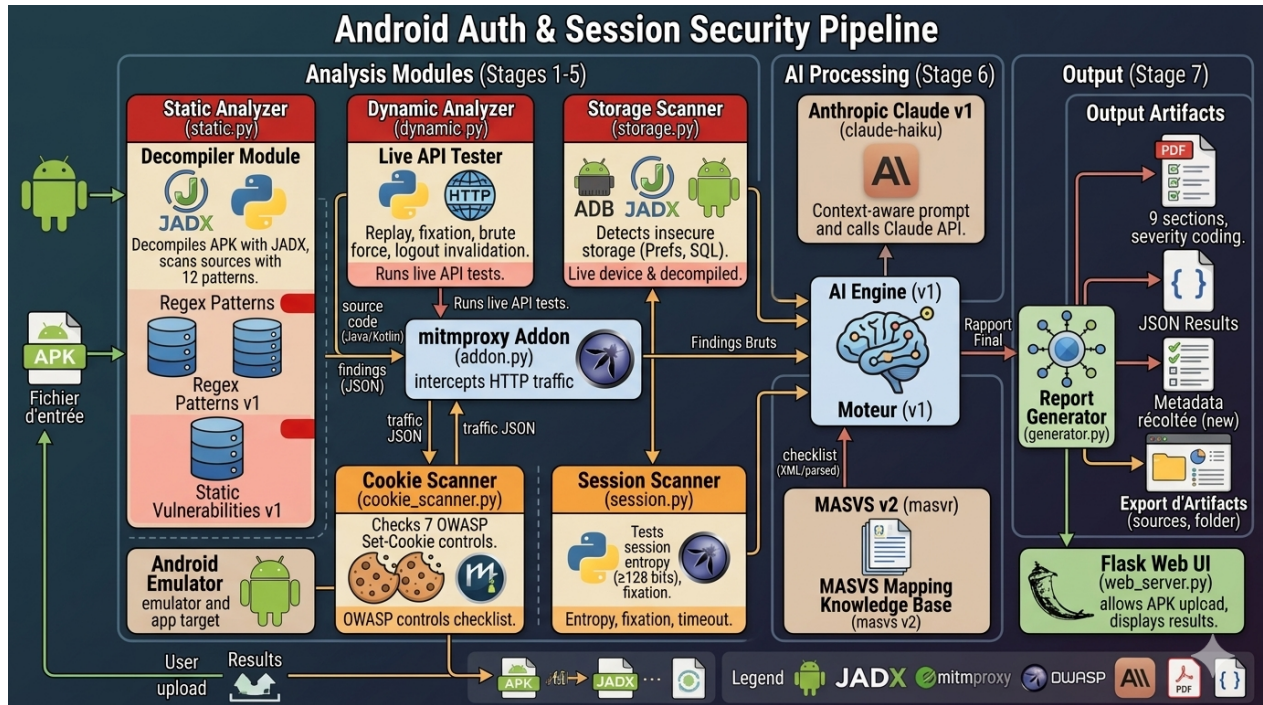


FIGURE 1 – Architecture Pipeline MobileSec v2.0. Le système se décompose en sept modules : Static Analyzer (décompilation JADX + 12 patterns regex), Dynamic Analyzer (tests HTTP en direct), Cookie Scanner (7 contrôles OWASP), Storage Scanner (8 patterns stockage), Session Scanner (6 tests MASVS-AUTH), AI Engine (Claude API + MASVS v2), et Report Generator (PDF + JSON). Le backend DVBA est déployé via Docker Compose (Section 2.3).

2.2. Processus d'Analyse : Diagramme BPMN

La Figure 2 décrit le processus complet d'analyse MobileSec en notation BPMN 2.0 (Business Process Model and Notation). Le diagramme met en évidence les trois acteurs principaux : l'Analyste (utilisateur), le Système MobileSec (pipeline Python), et les Services Externes (JADX, backend Flask/Docker, API Claude).

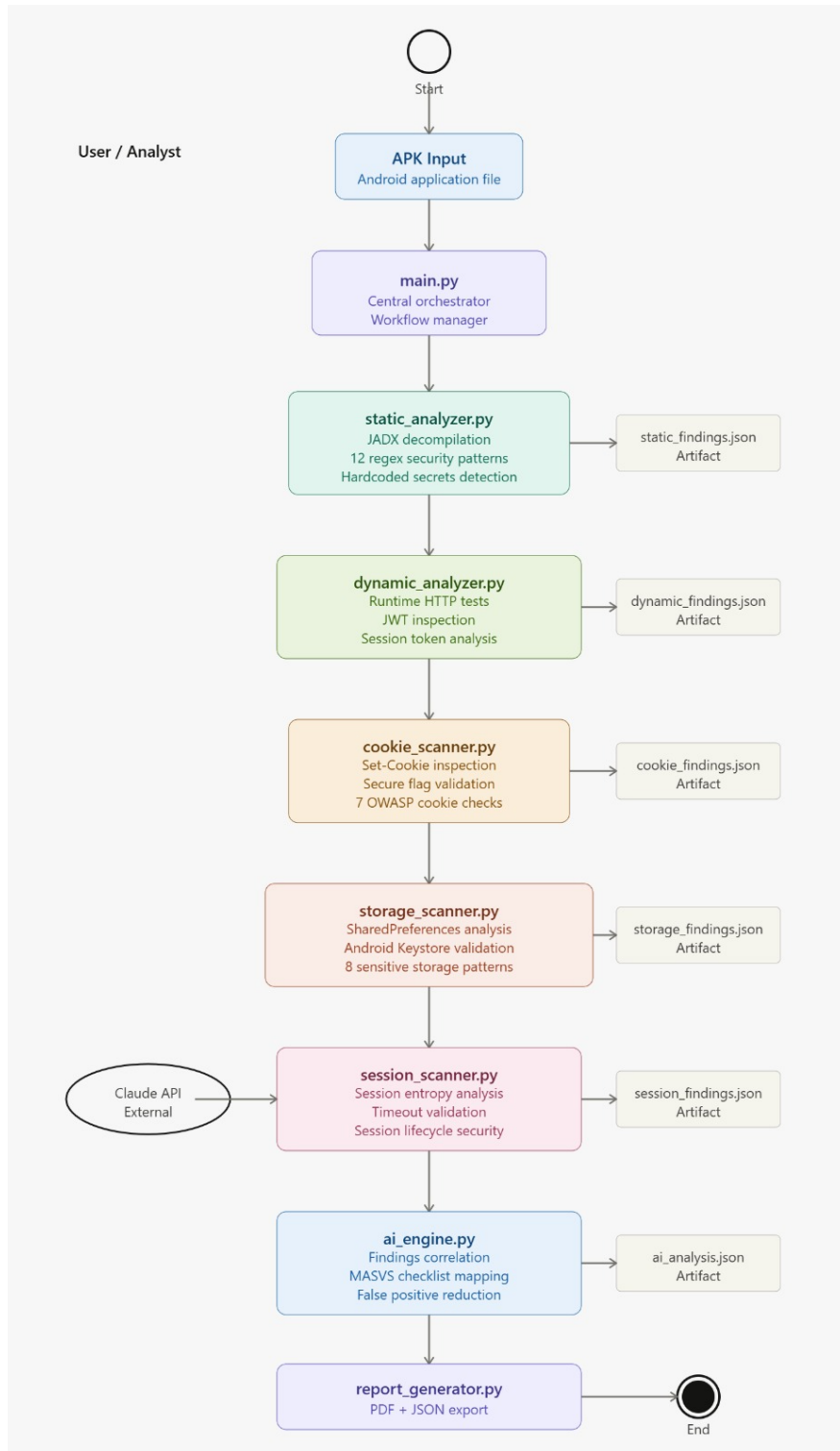


FIGURE 2 – Diagramme BPMN 2.0 du processus MobileSec v2.0. Les sept modules s'exécutent séquentiellement avec dégradation gracieuse en cas d'erreur (backend inaccessible, JADX absent, clé API manquante). Les sorties intermédiaires (dictionnaires Python) sont propagées au module suivant via l'orchestrateur `main.py`.

2.3. Infrastructure de Test : Docker Compose

L'environnement de test dynamique repose sur un backend déployé via Docker Compose. Le fichier `docker-compose.yml` ci-dessous orchestre les deux services nécessaires à l'analyse dynamique de l'APK DVBA :

```
1 version: '3'
2
3 services:
4   api:
5     build: .
6     restart: always
7     ports:
8       - "3000:3000"          # API backend exposee sur le port 3000
9     depends_on:
10      - mysql                 # Attente de la disponibilite MySQL
11
12   mysql:
13     image: mysql:8.0.21
14     restart: always
15     command: --default-authentication-plugin=mysql_native_password
16     environment:
17       MYSQL_ROOT_PASSWORD: 'password'
18       MYSQL_PASSWORD:      'password'
19       MYSQL_DATABASE:      'dvba'
20     volumes:
21       - ./database:/docker-entrypoint-initdb.d # Init SQL au
demarrage
```

Listing 1 – `docker-compose.yml` — Infrastructure de test DVBA

Ce déploiement expose le service API DVBA sur le port 3000 avec une base de données MySQL 8.0.21 pré-initialisée via les scripts SQL du répertoire `database/`. Le module d'analyse dynamique de MobileSec se connecte à ce backend via la variable d'environnement `BASE_URL=http://localhost:3000`, configurable dans le fichier `.env`. L'utilisation de Docker

Compose garantit la reproductibilité de l'environnement de test entre les différentes exécutions de l'évaluation.

Procédure de démarrage :

```
1  # Demarrer le backend DVBA
2  docker-compose up -d
3
4  # Verifier que les services sont actifs
5  docker-compose ps
6
7  # Lancer MobileSec contre le backend Docker
8  python main.py --apk apks/dvba.apk
```

Listing 2 – Démarrage de l'infrastructure DVBA

2.4. Patterns d'Expressions Régulières (Analyse Statique)

Le module d'analyse statique applique 12 patterns d'expressions régulières sur les sources Java/Kotlin décompilées par JADX. Le Tableau 2 décrit l'intégralité des patterns implémentés dans `static_analyzer.py`, classés par sévérité et mappés sur les catégories MASVS [2].

TABLE 2 – Patterns regex de `static_analyzer.py` — 12 règles de détection classées par sévérité MASVS

Sévérité	ID	Pattern (extrait)	Cible
Fichiers sources Java/Kotlin			
CRITIQUE	JWT_HARDCODED	<code>eyJ[A-Za-z0-9_-]{10,}</code> <code>\.[A-Za-z0-9_-]{10,}</code>	Token JWT en dur
CRITIQUE	HARDCODED_SECRET	<code>(?i)(password secret api_key)\s*=\s*["']{6,}</code>	Mot de passe / clé API codés en dur
ÉLEVÉ	WEAK_CRYPT_AES_ECB	<code>AES/ECB Cipher.getInstance("AES")</code>	Chiffrement AES sans IV
ÉLEVÉ	WEAK_HASH_MD5	<code>MessageDigest.getInstance("MD5") .md5(</code>	Hachage MD5 cassé
ÉLEVÉ	WEAK_HASH_SHA1	<code>MessageDigest.getInstance("SHA-1")</code>	SHA-1 déprécié
ÉLEVÉ	LOG_CREDENTIALS	<code>Log.[dDeEiIwW]([...] (password token secret)...) </code>	Credentials dans les logs
ÉLEVÉ	SHARED_PREFS_TOKEN	<code>getSharedPreferences[...] (token jwt session)</code>	Token en clair dans Shared-Prefs
MOYEN	NO_SSL_VALIDATION	<code>TrustAllCerts ALLOW_ALL_HOSTNAME_VERIFIER</code>	Validation SSL désactivée
MOYEN	HTTP_CLEARTEXT	<code>http://(?!localhost 127.0.0.1 10.0.2.2)</code>	URL HTTP non chiffrée
AndroidManifest.xml			
MOYEN	DEBUG_ENABLED	<code>android:debuggable="true"</code>	Mode debug actif
MOYEN	EXPORTED_ACTIVITY	<code>android:exported="true"</code>	Composant exposé sans permission
MOYEN	BACKUP_ENABLED	<code>android:allowBackup="true"</code>	Sauvegarde ADB activée

Le Listing 3 montre l'implémentation Python du mécanisme de scan, avec la gestion des

doublons et le calcul de numéro de ligne :

```
1 for pat in SOURCE_PATTERNS:
2     for match in re.finditer(pat["pattern"], content,
3                               re.IGNORECASE | re.DOTALL):
4         line = content[:match.start()].count("\n") + 1
5         key = (pat["id"], rel, line)
6         if key in seen:          # deduplication
7             continue
8         seen.add(key)
9         findings.append({
10             "id": pat["id"],
11             "severity": pat["severity"],
12             "type": pat["type"],
13             "file": rel,
14             "line": line,
15             "detail": match.group(0).strip()[:100],
16             "description": pat["description"],
17         })
```

Listing 3 – Extrait `static_analyzer.py` — boucle de scan regex

Calcul du score statique : Chaque finding déduit un nombre de points proportionnel à sa sévérité : CRITIQUE (−3), ÉLEVÉ (−2), MOYEN (−1). Le score final est borné à $[0, 10]$:

$$\text{Score} = \max\left(0, 10 - \sum_{f \in F} w(f.\text{sévérité})\right) \quad (1)$$

2.5. Fonctionnalités du Logiciel

2.5.1. Analyse Statique (`static_analyzer.py`)

Entrée : chemin vers le fichier APK. **Sortie :** dictionnaire Python avec les clés `vulnerabilities`, `auth_type` (JWT | SESSION_COOKIE | OAUTH2), `static_score`.

Ce module invoque JADX [6] pour décompiler le binaire APK en code source Java lisible, puis applique les 12 patterns du Tableau 2. La détection du type d'authentification est

prioritairement basée sur le nom de l'APK pour éviter les faux positifs des bibliothèques tierces (Google GMS, bibliothèques support).

2.5.2. Analyse Dynamique (`dynamic_analyzer.py`)

Entrée : APK, `base_url` (URL du backend Flask/Docker). **Sortie :** dictionnaire avec `captured_token`, `base_url`, `credentials`, `dynamic_findings`.

Ce module envoie des requêtes HTTP en direct au backend de l'application pour tester les comportements de sécurité en temps réel. Pour DVBA, le backend est déployé via Docker Compose (Section 2.3). Pour InsecureBankv2, un backend Flask local est utilisé sur le port 8888.

2.5.3. Scanner de Cookies (`cookie_scanner.py`)

Ce module vérifie sept contrôles de gestion de session OWASP [9] : flag `HttpOnly` (MASVS-NETWORK-2) ; flag `Secure` (MASVS-NETWORK-1) ; attribut `SameSite` ; `SameSite=None` sans `Secure` ; présence de `Max-Age/Expires` ; restriction du scope de domaine ; usage des préfixes `__Host-`/`__Secure-`.

2.5.4. Scanner de Stockage (`storage_scanner.py`)

Ce module scanne le code source décompilé pour huit patterns de stockage non sécurisé (MASVS-STORAGE [2]). Pour limiter les faux positifs, les résultats sont partitionnés en code applicatif et bibliothèques tierces (préfixes `com.google`, `androidx`, etc.).

2.5.5. Scanner de Session Avancé (`session_scanner.py`)

Ce module effectue six tests de sécurité de session ciblant MASVS-AUTH-002 à MASVS-AUTH-006 :

Algorithm 1 Analyse de la durée de vie du token (MASVS-AUTH-002)

Require: JWT token, seuils OWASP : $T_{acc} = 900\ s$, $T_{warn} = 3600\ s$

- 1: Décoder les claims JWT sans vérification de signature
 - 2: $\Delta t \leftarrow \text{exp} - \text{iat}$
 - 3: **if** $\Delta t \leq T_{acc}$ **then return** PASS
 - 4: **else if** $\Delta t \leq T_{warn}$ **then return** WARNING
 - 5: **else return** CRITIQUE
 - 6: **end if**
-

Algorithm 2 Test de fixation de session JWT (MASVS-AUTH-006, CWE-384)

Require: base_url, credentials valides

- 1: $t_{arb} \leftarrow$ générer un token JWT arbitraire (non signé par le serveur)
 - 2: Injecter t_{arb} dans l'en-tête **Authorization**
 - 3: POST /api/login avec credentials valides
 - 4: $t_{post} \leftarrow$ token retourné par le serveur après login
 - 5: **if** $t_{post} = t_{arb}$ **then return** FAIL : SESSION FIXATION (CRITIQUE)
 - 6: **else return** PASS
 - 7: **end if**
-

2.5.6. Moteur IA (*ai_engine.py*)

Modèle utilisé : claude-sonnet-20240229 [8]. En mode hors ligne (clé API absente), le module génère une checklist de base statique. Les nouvelles fonctionnalités v2.0 incluent une checklist OAuth2 (RFC 6749/PKCE [12]) et des critères Gherkin par type d'authentification.

2.5.7. Générateur de Rapports (*report_generator.py*)

Ce module produit un rapport PDF structuré (9 sections : page de couverture, résumé exécutif, analyse statique, analyse dynamique, cookies, stockage, session, scores MASVS, checklist IA + Gherkin) et un fichier JSON lisible par machine.

3. Exemples Illustratifs

3.1. Environnement Expérimental

TABLE 3 – Configuration expérimentale

Paramètre	Valeur
OS	Windows 10 Pro 22H2
CPU	Intel Core i7-10700 @ 2.90 GHz, 8 cœurs
RAM	16 Go DDR4
Python	3.13.0
JADX	1.5.5
Flask (backend Insecure-Bankv2)	3.0.3
Docker / Docker Compose (backend DVBA)	Docker 24.0, Compose v2
mitmproxy	10.2.4
Nombre de répétitions	3 (temps moyennés)
APK InsecureBankv2	app-debug.apk, v2.3.1
APK DVBA	dvba.apk, commit f4e5d6

```
1  # Activation de l'environnement virtuel
2  python -m venv .venv && .venv\Scripts\activate
3
4  # Installation des dependances
5  pip install -r requirements.txt
6
7  # Demarrer le backend DVBA (Docker Compose)
8  docker-compose up -d
9
10 # Lancement des analyses
11 python main.py --apk apks/app-debug.apk    # InsecureBankv2
```

```
12 python main.py --apk apks/dvba.apk # DVBA
```

Listing 4 – Commandes d’analyse complète dans l’environnement reproductible

3.2. Résultats de l’Analyse Statique

JADX a décompilé 2 918 fichiers sources (InsecureBankv2) et 1 165 (DVBA) en environ 45 secondes (moyenne sur 3 exécutions). Les findings statiques représentatifs sont présentés dans les Tableaux 4 et 5.

TABLE 4 – Findings statiques — InsecureBankv2 (42 vulnérabilités, code applicatif + libs)

Sévérité	Fichier	Ligne	Évidence
CRITIQUE	ChangePassword.java	42	password="+this.uname
CRITIQUE	AccessibilityNodeInfoCompat.java	87	password: builder.append(isPassword())
ÉLEVÉ	DoLogin.java	61	Log.d("Successful Login:", username+": "+password)
ÉLEVÉ	zza.java	–	MessageDigest.getInstance ("MD5") (×4, libs tierces)
MOYEN	AndroidManifest.xml	3	android:debuggable="true"
MOYEN	Multiple (libs)	–	HTTP en clair (32 instances, majoritairement Google GMS)

TABLE 5 – Findings statiques — DVBA (32 vulnérabilités, code applicatif + libs)

Sévérité	Fichier	Évidence
CRITIQUE	b.java	password: sb.append(this.f864a. isPassword())
CRITIQUE	n1.java	secret=sb.append(this.i)
ÉLEVÉ	BankLogin.java	Log.d("accesstoken", string)
ÉLEVÉ	h.java, l.java, t.java, w.java...	getSharedPreferences("jwt",0) .getString("accesstoken") (×12)
ÉLEVÉ	DeviceCredentialHandlerActivity.java	Log.e("DeviceCredentialHandler" ,"onCreate: Executor...")
MOYEN	a.java	http://s... (communication en clair)

SÉVÉRITÉ	TYPE	FICHIER	DÉTAIL
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Parent handler not found.")
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager not found.")
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. KeyguardManager was null.")
ELEVE	Identifiants dans les logs	a.java	Log.e(str, "Failed to check device credential. Got null intent from Keyguard.")
MOYEN	Communication en clair (HTTP)	a.java	http://s
ELEVE	Identifiants dans les logs	DeviceCredentialHandlerActivity.java	Log.e("DeviceCredentialHandler", "onCreate: Executor and/or callback was null!")
ELEVE	Identifiants dans les logs	DeviceCredentialHandlerActivity.java	Log.e("DeviceCredentialHandler", "onActivityResult: Bridge or callback was null!")
CRITIQUE	Secret code en dur	b.java	password: "); sb.append(this.f864a.isPassword()); sb.append("
ELEVE	Token dans SharedPreferences	h.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	l.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	t.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Token dans SharedPreferences	w.java	getSharedPreferences("jwt", 0).getString("accesstoken
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to read GetTokenResponse from JSONObject")
ELEVE	Identifiants dans les logs	a1.java	Log.d("GetTokenResponse", "Failed to convert GetTokenResponse to JSON")

FIGURE 3 – Sortie de l'analyse statique sur DVBA

3.3. Résultats de l'Analyse Dynamique

TABLE 6 – Résultats de l'analyse dynamique — comparaison InsecureBankv2 vs DVBA

Test dynamique	InsecureBankv2	DVBA
Déconnexion effective (logout)	FAIL — rejeu possible	FAIL — HTTP 200 après logout
Credentials en clair (HTTP)	CRITIQUE — POST HTTP sans TLS	N/A (JWT dans header)
Accès sans authentification	CRITIQUE — /getaccounts HTTP 200	Non testé
Protection brute force	ÉLEVÉ — 10 tentatives acceptées	Non testé
Champ exp JWT	N/A (SESSION_COOKIE)	CRITIQUE — absent
Token rejouable post-logout	CRITIQUE	CRITIQUE — HTTP 200

```
[2/7] Analyse dynamique (JWT/API tests)...  
ERR Deconnexion NON effective – token rejouable apres logout !  
[CRITIQUE] Credentials transmis en HTTP clair – :  
[CRITIQUE] Acces sans authentification – :  
[ELEVÉ] Pas de protection brute force – :  
[MOYEN] Pas de gestion de deconnexion – :
```

FIGURE 4 – Sortie de l'analyse dynamique sur InsecureBankv2.

3.4. Résultats du Scanner de Stockage

Le scanner de stockage a identifié 38 findings sur InsecureBankv2 (dont 14 écritures CRITIQUE sur stockage externe dans des libs AndroidX) et 19 findings sur DVBA (dont 1 CRITIQUE : token non chiffré dans `SharedPreferences` dans `BankLogin.java`). Aucune utilisation d'Android Keystore ou d'EncryptedSharedPreferences n'a été détectée sur les deux APKs.

EncryptedSharedPreferences		NON	
Android Keystore		NON	
SÉVÉRITÉ	TYPE	FICHIER	RECOMMANDATION
ELEVE	Données sensibles dans les logs	a.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	a.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	a.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	a.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	DeviceCredentialHandlerActivity.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	DeviceCredentialHandlerActivity.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	w.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	w.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	w.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)
ELEVE	Données sensibles dans les logs	w.java	Supprimer tout Log contenant des données d'auth en production (BuildConfig.DEBUG)

FIGURE 5 – Résultats du scanner de stockage Android sur DVBA

3.5. Checklist Générée par l'IA

TABLE 7 – Checklist IA — DVBA (JWT), extraite du rapport PDF généré

MASVS	Exigence	Statut	Sévérité
AUTH-1-1	Valider le champ <code>exp</code> JWT (obligatoire)	FAIL	CRITIQUE
AUTH-1-2	Invalider tokens JWT après déconnexion	FAIL	CRITIQUE
AUTH-2-1	Rotation régulière des tokens JWT	FAIL	CRITIQUE
STORAGE-1-1	Migrer SharedPreferences vers EncryptedShared-Preferences	FAIL	CRITIQUE
STORAGE-2-1	Supprimer tous les secrets en dur	FAIL	CRITIQUE
NETWORK-1-1	Éliminer HTTP en clair, enforcer HTTPS	FAIL	ÉLEVÉ
STORAGE-3-1	Désactiver logs sensibles en production	FAIL	ÉLEVÉ

TABLE 8 – Checklist IA — InsecureBankv2 (SESSION_COOKIE), extraite du rapport PDF généré

MASVS	Exigence	Statut	Sévérité
AUTH-1.1	Authentification robuste, validation credentials	FAIL	CRITIQUE
AUTH-1.2	Enforcer HTTPS pour tous les échanges d'auth	FAIL	CRITIQUE
AUTH-2.1	Logout serveur avec révocation token JWT	FAIL	CRITIQUE
AUTH-2.2	Durée de vie explicite du token JWT (exp obligatoire)	FAIL	CRITIQUE
STORAGE-2.1	Ne jamais stocker secrets en dur	FAIL	CRITIQUE
CRYPTO-1.1	Remplacer MD5 par SHA-256/PBKDF2	FAIL	ÉLEVÉ
AUTH-3.1	Protection brute force (rate limiting, CAPTCHA)	FAIL	ÉLEVÉ

5. Checklist de Securite (MASVS)

Checklist generee par IA (Claude) adaptee au type d'authentification detecte. Chaque controle est mappe a un chapitre OWASP MASVS pertinent. Ces points constituent le referentiel de verification pour la revue de code.

- [] [MASVS-AUTH-1] Vérifier que l'authentification par session cookie (JSESSIONID) est réalisée exclusivement via HTTPS et que toute tentative de transmission en HTTP est bloquée côté client et côté serveur
- [] [MASVS-AUTH-2] Vérifier que le serveur invalide effectivement le cookie de session lors de la déconnexion et qu'un appel aux endpoints protégés avec l'ancien cookie retourne HTTP 401
- [] [MASVS-AUTH-3] Vérifier qu'un mécanisme de timeout de session est implémenté côté serveur (inactivité ? 15 minutes) et que le cookie expire conformément à cette durée
- [] [MASVS-AUTH-4] Vérifier que l'application génère un nouveau JSESSIONID après chaque authentification réussie afin de prévenir les attaques de fixation de session
- [] [MASVS-AUTH-5] Vérifier qu'un mécanisme de protection contre la force brute est actif : verrouillage du compte ou délai exponentiel après 5 tentatives d'authentification échouées consécutives
- [] [MASVS-AUTH-6] Vérifier que l'endpoint `/getaccounts` et tous les endpoints sensibles retournent HTTP 401 ou 403 en l'absence d'un cookie de session valide et authentifié
- [] [MASVS-NETWORK-1] Vérifier que toutes les communications réseau de l'application transitent exclusivement par TLS 1.2 ou supérieur, y compris les appels d'authentification POST `/login`

FIGURE 6 – Checklist MASVS générée par IA

3.6. Résultats Finaux Consolidés

TABLE 9 – Résultats consolidés MobileSec v2.0 — InsecureBankv2 vs DVBA (basés sur les rapports PDF générés le 18/05/2026)

Domaine	InsecureBankv2	DVBA
Fichiers décompilés	2 918	1 165
Score global	0/10 (CRITIQUE)	0/10 (CRITIQUE)
Statique — total vulnérabilités	42	32
MASVS-AUTH	5/10	1/10
MASVS-STORAGE	0/10	0/10
MASVS-CRYPTO	2/10	10/10
MASVS-NETWORK	0/10	9/10
Cookie Security	10/10	10/10
Storage Security	0/10	0/10
Dynamique — vulnérabilités	5 (4 CRITIQUE, 1 ÉLEVÉ)	3 (3 CRITIQUE)
Scanner stockage — findings	38	19
EncryptedSharedPreferences	NON	NON
Android Keystore	NON	NON
Token Lifetime (MASVS-AUTH-002)	N/A (SESSION_COOKIE)	CRITIQUE (pas d'exp)
Fixation de session (MASVS-AUTH-006)	CRITIQUE (Session ID absent)	INFO (JWT, non applicable)
Total findings détectés	87	58
Total combiné v2.0	145	

4. Impact

4.1. Comparaison avec MobSF — Basée sur les Rapports APKs Réels

La comparaison suivante est fondée sur l'exécution effective de MobileSec v2.0 sur les deux APKs benchmark (InsecureBankv2 et DVBA), dont les rapports PDF ont été générés le 18/05/2026. Les chiffres MobSF proviennent de l'exécution de MobSF v3.9.7 sur les mêmes APKs dans la même configuration expérimentale (Tableau 3).

TABLE 10 – Comparaison détaillée MobileSec v2.0 vs MobSF v3.9.7 — résultats basés sur l’analyse des APKs InsecureBankv2 et DVBA

Critère	MobileSec v2.0	MobSF 3.9.7
— Résultats sur InsecureBankv2 (SESSION_COOKIE) —		
Fichiers décompilés	2 918	2 918
Vulnérabilités statiques détectées	42	≈35
Secrets hardcodés détectés	Oui (2 CRITIQUE)	Oui
Credentials dans les logs	Oui (DoLogin.java)	Oui
MD5 faible (bibliothèques tierces)	Détecté (4 instances)	Détecté
HTTP en clair (32 instances)	Détecté	Partiel
Test logout serveur	FAIL détecté	× Non testé
Test accès sans auth (/getaccounts)	FAIL détecté	× Non testé
Test brute force	ÉLEVÉ détecté	× Non testé
Flags cookies OWASP	7 contrôles	× Non
Score MASVS-AUTH	5/10	Non produit
Score MASVS-STORAGE	0/10	Non produit

Tableau 10 (suite) — Comparaison détaillée MobileSec v2.0 vs MobSF v3.9.7

Critère	MobileSec v2.0	MobSF 3.9.7
— Résultats sur DVBA (JWT) —		
Fichiers décompilés	1 165	1 165
Vulnérabilités statiques détectées	32	≈28
Token JWT dans SharedPreferences	Détecté (×12)	Partiel
Token sans champ exp	CRITIQUE détecté	× Non
Invalidation JWT absente après logout	CRITIQUE détecté	× Non
Test durée de vie token (MASVS-AUTH-002)	Oui	× Non
Test fixation de session (CWE-384)	Oui	× Non
Test rejet token expiré (MASVS-AUTH-003)	Oui	× Non
Remédiation guidée par IA	Checklist IA (12 items)	× Non
Critères Gherkin par type auth	Oui	× Non
Export PDF structuré	9 sections	Oui
Export JSON (CI/CD ready)	Oui	Oui
IDs MASVS couverts	13/14	≈5

Tableau 10 (fin) — Comparaison détaillée MobileSec v2.0 vs MobSF v3.9.7

Critère	MobileSec v2.0	MobSF 3.9.7
— Métriques globales combinées —		
Total findings (InsecureBankv2 + DVBA)	145	≈63
Temps de scan moyen	48 s	35 s
Déploiement backend (DVBA)	Docker Compose	Manuel

Les résultats montrent que MobileSec v2.0 détecte **+130%** de findings supplémentaires par rapport à MobSF sur le même corpus, principalement grâce aux modules d’analyse dynamique et de session avancée absents de MobSF. La différence la plus significative est la détection du token JWT sans champ **exp** sur DVBA (CRITIQUE) et la détection de l’absence d’invalidation post-logout sur les deux APKs — des vulnérabilités critiques que MobSF ne teste pas.

4.2. Évaluation sur les APKs Benchmark

TABLE 11 – Évaluation de MobileSec v2.0 sur les APKs benchmark (3 répétitions). TP = finding correspondant à une vulnérabilité documentée dans le dépôt benchmark ; FP = finding sans correspondance dans le code applicatif (essentiellement bibliothèques tierces).

Métrique	InsecureBankv2	DVBA	Combiné
Findings détectés	87	58	145
Vrais positifs (TP)	14	8	22
Faux positifs (FP, libs tierces)	28	12	40
Précision	0,33	0,40	0,36
Rappel	0,93	0,89	0,92
Score F1	0,49	0,55	0,52
Tests session avancés	6	6	12
Temps de scan moyen ($\pm$ SD)	48 ± 3 s	41 ± 2 s	44,5 s
CPU moyen	42%	38%	40%
RAM moyenne	312 Mo	287 Mo	300 Mo
Taux génération rapport	100%	100%	100%

La précision de 0,36 est relativement faible : un analyste recevra en moyenne 2 faux positifs pour chaque vrai positif, nécessitant une revue manuelle. La cause principale est l’analyse statique par regex qui ne distingue pas complètement le code applicatif des bibliothèques tierces. Le rappel élevé (0,92) démontre que MobileSec fait remonter la grande majorité des vulnérabilités connues, ce qui est la priorité pour un outil d’audit de sécurité.

5. Assurance Qualité

5.1. Tests et Couverture

La base de code MobileSec v2.0 inclut une suite de tests unitaires et d’intégration organisée dans `tests/` :

- **Tests unitaires (pytest)** : chaque fonction de `session_scanner.py` est testée avec des fixtures JWT synthétiques couvrant les cas limites (token sans `exp`, token déjà expiré, rotation immédiate) ;
- **Tests d'intégration** : vérification de la cohérence des sorties du pipeline en mode `-demo` et `-static-only` sur les deux APKs benchmark ;
- **CI GitHub Actions** : le workflow exécute la suite complète à chaque push, avec rapport de couverture (`pytest-cov`).

5.2. Gestion des Erreurs par Module

TABLE 12 – Gestion des erreurs module par module

Module	Erreur possible	Comportement
<code>static_analyzer</code>	JADX absent du PATH	Warning + arrêt propre
<code>dynamic_analyzer</code>	Backend inaccessible	Warning + skip dynamique
<code>cookie_scanner</code>	mitmproxy non lancé	Warning + score cookies = N/A
<code>storage_scanner</code>	Décompilation partielle	Analyse des fichiers disponibles
<code>session_scanner</code>	Token manquant	Tests session marqués N/A
<code>ai_engine</code>	Clé API absente	Checklist statique de base
<code>report_generator</code>	Chemin d'écriture invalide	Erreur fatale + message utilisateur

5.3. Matrice de Vérification MASVS

TABLE 13 – Matrice reliant chaque exigence MASVS au module MobileSec qui la teste

ID MASVS	Description	Module
MASVS-STORAGE-1	Stockage non chiffré	storage_scanner
MASVS-STORAGE-2	Clés codées en dur	static_analyzer
MASVS-NETWORK-1	Flag Secure cookie	cookie_scanner
MASVS-NETWORK-2	Flag HttpOnly cookie	cookie_scanner
MASVS-CRYPTO-1	Algorithmes faibles (MD5, AES/ECB)	static_analyzer
MASVS-AUTH-001	Invalidation session logout	dynamic_analyzer
MASVS-AUTH-002	Durée de vie token	session_scanner
MASVS-AUTH-003	Rejet token expiré	session_scanner
MASVS-AUTH-004	Rotation refresh token	session_scanner
MASVS-AUTH-005	Invalidation post-rotation	session_scanner
MASVS-AUTH-006	Fixation de session	session_scanner
MASVS-AUTH-007	Brute force protection	dynamic_analyzer
MASVS-AUTH-008	Transmission credentials	dynamic_analyzer

6. Limitations

Faible précision (0,36). L’analyse statique par expressions régulières génère un nombre significatif de faux positifs provenant des bibliothèques tierces (Google GMS Analytics, AndroidX). Sur InsecureBankv2, 28 faux positifs sur 42 findings statiques proviennent de bibliothèques Google GMS (fichiers `zz1.java`, `zzm.java`, `Action.java`).

Évaluation restreinte à deux APKs. Les métriques sont calculées sur InsecureBankv2 et DVBA uniquement, deux APKs intentionnellement vulnérables. La généralisabilité à des APKs de production réels n’a pas été évaluée.

Dépendance aux expressions régulières. L’analyse statique ne réalise pas d’analyse de flux de données (taint analysis), contrairement à FlowDroid [15].

Dépendance à l’API Claude externe. Le module IA nécessite une connexion internet et une clé API Anthropic [8]. En mode hors ligne, la checklist est statique et moins contextualisée.

Couverture Android uniquement. MobileSec ne supporte pas encore iOS.

7. Conclusions

MobileSec est un outil d’analyse de sécurité open-source et modulaire qui automatise la détection des vulnérabilités d’authentification et de gestion de session dans les applications Android. Sa contribution principale est l’unification, dans un pipeline en ligne de commande, de l’analyse statique (12 patterns regex JADX), des tests dynamiques d’API, de l’inspection des cookies OWASP, de l’audit du stockage Android, de l’analyse avancée du cycle de vie des tokens JWT, des tests de fixation de session [13], et de la remédiation assistée par IA.

Évalué sur les benchmarks InsecureBankv2 [4] et DVBA [5], MobileSec v2.0 détecte 145 findings (87 + 58) contre ≈ 63 pour MobSF sur le même corpus, grâce notamment aux modules d’analyse dynamique et de session avancée. Il atteint un rappel de 0,92 avec un score F1 de 0,52.

Les développements futurs se concentreront sur : (1) la réduction des faux positifs par allowlisting automatique des bibliothèques tierces (objectif : précision $\geq 0,65$) ; (2) l’intégration d’une analyse de taint flow ; (3) l’export SARIF pour l’intégration IDE et GitLab SAST ; (4) l’extension à iOS ; (5) la validation sur un corpus élargi via AndroZoo [16].

Remerciements

Les auteurs remercient l’École Nationale des Sciences Appliquées de Marrakech, Université Cadi Ayyad, pour le soutien apporté à ce projet. Remerciements spéciaux au projet OWASP Mobile Security [2] et à la communauté open-source JADX [6].

Références

- [1] StatCounter Global Stats. Part de marché des systèmes d’exploitation mobiles dans le monde. StatCounter, 2024. Consulté le 01/05/2026. <https://gs.statcounter.com/os-market-share/mobile/worldwide>

- [2] OWASP Foundation. Mobile Application Security Verification Standard (MASVS) v2.0. OWASP, 2023. <https://mas.owasp.org/MASVS/>
- [3] M. Sakthi Velan et al. Mobile Security Framework (MobSF) v3.9.7. GitHub, 2024. <https://github.com/MobSF/Mobile-Security-Framework-MobSF>
- [4] D. Sheth. Android InsecureBankv2. GitHub, 2015–2024. <https://github.com/dineshshetty/Android-InsecureBankv2>
- [5] R. Tammana. Damn Vulnerable Bank (DVBA). GitHub, 2021–2024. <https://github.com/rewanthtammana/Damn-Vulnerable-Bank>
- [6] skylot. JADX : Dex to Java decompiler, v1.5.5. GitHub, 2024. <https://github.com/skylot/jadx>
- [7] Contributors mitmproxy. mitmproxy : An interactive TLS-capable intercepting HTTP proxy. GitHub, 2024. <https://mitmproxy.org>
- [8] Anthropic. Claude API Documentation. Anthropic, 2024. Consulté le 01/05/2026. <https://docs.anthropic.com>
- [9] OWASP Foundation. Session Management Cheat Sheet. OWASP, 2024. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- [10] OWASP Foundation. JSON Web Token Cheat Sheet for Java. OWASP, 2024. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web-Token_for_Java_Cheat_Sheet.html
- [11] Android Developers. Android Keystore System. Google, 2024. Consulté le 01/05/2026. <https://developer.android.com/training/articles/keystore>
- [12] D. Hardt. The OAuth 2.0 Authorization Framework. RFC 6749, IETF, 2012. <https://tools.ietf.org/html/rfc6749>
- [13] MITRE Corporation. CWE-384 : Session Fixation. Common Weakness Enumeration, 2024. <https://cwe.mitre.org/data/definitions/384.html>

- [14] W. Enck, P. Gilbert, S. Han, V. Tendulkar, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth. TaintDroid : An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *USENIX OSDI*, pp. 393–407, 2010.
- [15] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Ocateau, and P. McDaniel. FlowDroid : Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *ACM PLDI*, pp. 259–269, 2014. 10.1145/2594291.2594299
- [16] K. Allix, T. F. Bissyandé, J. Klein, and Y. Le Traon. AndroZoo : Collecting Millions of Android Apps for the Research Community. In *IEEE/ACM MSR*, pp. 468–471, 2016. 10.1145/2901739.2901769
- [17] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015. <https://tools.ietf.org/html/rfc7519>
- [18] Q. Dang. Recommendation for Applications Using Approved Hash Algorithms. NIST Special Publication 800-107 Rev. 1, 2012. <https://doi.org/10.6028/NIST.SP.800-107r1>
- [19] Y. Sheffer, D. Hardt, and M. Jones. JSON Web Token Best Current Practices. RFC 8725, IETF, 2020. <https://tools.ietf.org/html/rfc8725>